ENTREGA 4: GATEWAY MQTT EM NODE.JS

Sensor de Alarme com Detector de Movimento (PIR)

Eduardo Octavio de Paula Souza

1. INTRODUÇÃO

Esta entrega documenta a implementação do gateway em Node.js, responsável por atuar como cliente MQTT e realizar o processamento intermediário das mensagens publicadas pelo ESP32. O gateway conecta-se ao mesmo broker Mosquitto já utilizado nas entregas anteriores, assina os tópicos do projeto, identifica o dispositivo de origem das mensagens e valida sua estrutura em JSON.

2. AMBIENTE DE DESENVOLVIMENTO

• Node.js

• Biblioteca mqtt (npm), utilizada como cliente MQTT em JavaScript.

• Broker MQTT Mosquitto, já implantado via Docker Compose.

3. TÓPICOS ASSINADOS PELO GATEWAY

```
alarme/sensor/pir        -> telemetria do sensor PIR (ESP32 -> gateway)
alarme/comando/sistema   -> comando de ativacao/desativacao
```

4. VALIDAÇÃO DAS MENSAGENS

Uma mensagem de telemetria é considerada válida quando o payload é um JSON sintaticamente correto e contém: deviceId (texto), sensorPIR (booleano) e timestamp (número). Mensagens que não atendam a esses critérios são registradas como inválidas, junto ao motivo da rejeição, sem interromper a execução do gateway.

5. CÓDIGO-FONTE (GATEWAY.JS)

```
const mqtt = require('mqtt');

const BROKER_URL = 'mqtt://192.168.2.168:1883';
const TOPICO_TELEMETRIA = 'alarme/sensor/pir';
const TOPICO_COMANDO = 'alarme/comando/sistema';
```

```
const client = mqtt.connect(BROKER_URL, {
  clientId: 'gateway-alarme-01'
});

client.on('connect', () => {
  console.log('Gateway conectado ao broker MQTT.');
  client.subscribe([TOPICO_TELEMETRIA, TOPICO_COMANDO], (err) => {
    if (!err) {
      console.log('Assinatura realizada nos topicos definidos.');
    }
  });
});

client.on('message', (topico, payload) => {
  const mensagemBruta = payload.toString();

  if (topico === TOPICO_TELEMETRIA) {
    processarTelemetria(mensagemBruta);
  } else if (topico === TOPICO_COMANDO) {
    console.log(`[COMANDO] ${mensagemBruta}`);
  }
});

function processarTelemetria(mensagemBruta) {
  let dados;
  try {
    dados = JSON.parse(mensagemBruta);
  } catch (erro) {
    registrarInvalida(mensagemBruta, 'JSON malformado');
    return;
  }

  const erros = validarCampos(dados);
  if (erros.length > 0) {
    registrarInvalida(mensagemBruta, erros.join('; '));
    return;
  }

  registrarValida(dados);
}

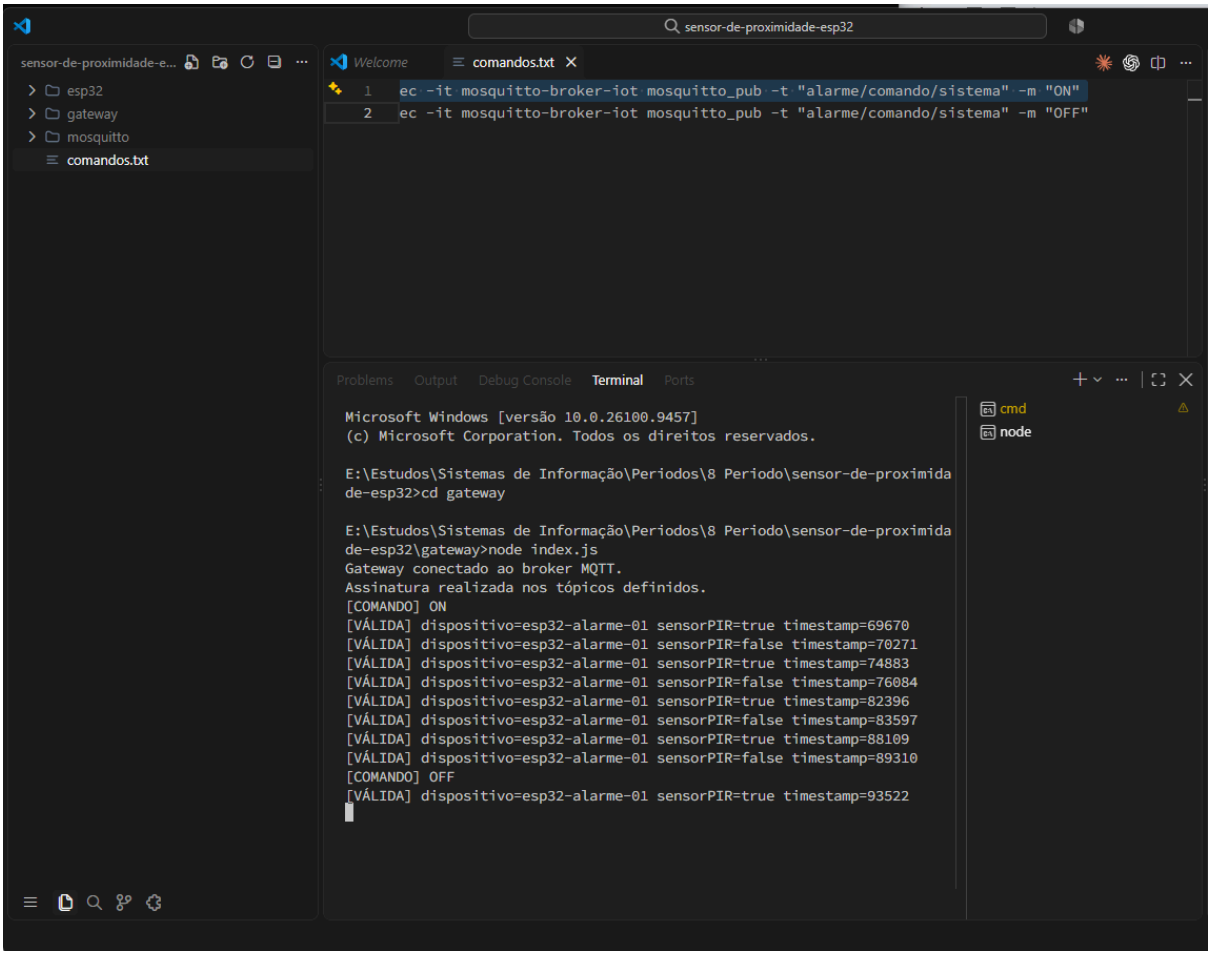
function validarCampos(dados) {
  const erros = [];
  if (typeof dados.deviceId !== 'string' || dados.deviceId.length === 0) {
    erros.push('campo deviceId ausente ou invalido');
  }
  if (typeof dados.sensorPIR !== 'boolean') {
    erros.push('campo sensorPIR ausente ou nao booleano');
  }
  if (typeof dados.timestamp !== 'number') {
    erros.push('campo timestamp ausente ou nao numerico');
  }
  return erros;
}
```

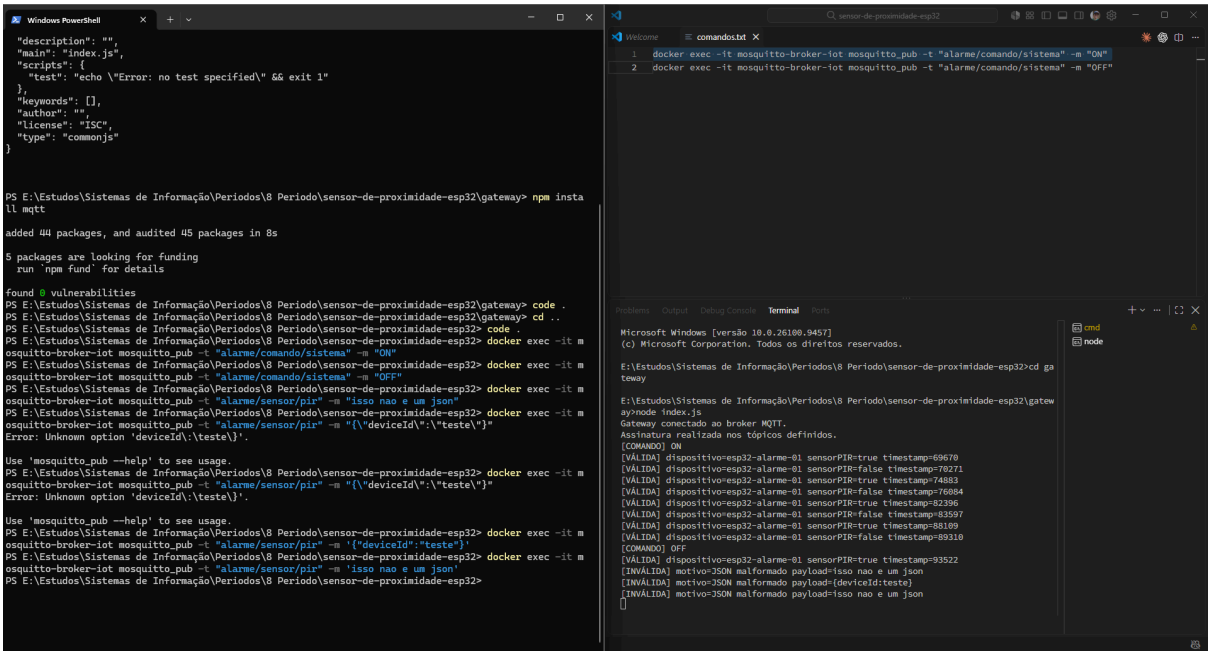
```
function registrarValida(dados) {
  console.log(
    `[VALIDA] dispositivo=${dados.deviceId} ` +
    `sensorPIR=${dados.sensorPIR} timestamp=${dados.timestamp}`
  );
}

function registrarInvalida(mensagemBruta, motivo) {
  console.log(`[INVALIDA] motivo=${motivo} payload=${mensagemBruta}`);
}
```

A identificação do dispositivo é realizada a partir do campo deviceId presente no corpo da mensagem, já que o projeto atual opera com um único dispositivo publicando em tópico fixo.

6. EVIDÊNCIA DE FUNCIONAMENTO





7. CONSIDERAÇÕES FINAIS

Esta entrega introduziu a camada de processamento intermediário da arquitetura, deslocando o foco do dispositivo embarcado para o tratamento das mensagens no lado do servidor. A validação estrutural das mensagens recebidas evita que dados malformados avancem para as próximas etapas de persistência em Redis e exibição no sistema web.